



MİMAR SİNAN GÜZEL SANATLAR ÜNİVERSİTESİ

**Uydu Görüntülerinde Çoklu Nesne İçeren Veri Setlerinin Sınıflandırma
İçin Ön İşlenmesi ve CNN Tabanlı Analizi**

Ad Soyad: Sıla ALAYI

Okul No: 20221101036

İçindekiler

1. GİRİŞ	2
2. YÖNTEM	2
2.1 Veri Seti	2
2.2 Veri Ön İşleme Süreci	2
2.3 Konvolüsyonel Sinir Ağı (CNN) Mimarisi.....	6
3. MODEL UYGULAMALARI, PERFORMANS ANALİZİ VE BULGULAR	7
3.1 Model 1	7
3.2 Model 2	8
3.2.1 Veri Seti Dengeleme ve Arttırma.....	8
3.2.2 Eğitim ve Performans Analizi	10
3.3 Model 3	10
3.4 Model 4	11
3.5 Model 5	12
3.6 Model 6	13
4. TRANSFER ÖĞRENME YAKLAŞIMI İLE PERFORMANS ANALİZİ.....	13
4.1 Ham Veri Seti ile VGG16.....	15
4.2 Dengelenmiş ve Arttırılmış Veri Seti ile VGG16.....	15
5. SONUÇ	16
6. KAYNAKÇA	17

1. GİRİŞ

Günümüzde uzaktan algılama teknolojileri, yüksek çözünürlüklü uydu görüntülerinin temin edilmesinde devrim yaratmıştır. Özellikle savunma sanayii, lojistik takip ve şehir planlama gibi alanlarda, uydu verileri üzerinden nesne tespiti yapmak kritik bir karar destek mekanizması haline gelmiştir. Bu bağlamda, askeri ve sivil hava araçlarının uydu görüntüleri üzerinden otomatik olarak tanınması, stratejik öneme sahip bir araştırma konusudur.

Mevcut literatürde nesne tespiti için kullanılan derin öğrenme tabanlı algoritmalar genellikle sahnenin tamamını analiz etmeye odaklanır. Bu çalışmada, nesne tespiti yerine, sınırlayıcı kutu bilgileri kullanılarak nesnelerin tekil görüntüler haline getirildiği ve ardından özelleştirilmiş bir Konvolüsyonel Sinir Ağı (CNN) ile sınıflandırıldığı bir metodoloji baz alınarak modellerinin başarısı test edilmek istenmiştir.

2. YÖNTEM

2.1 Veri Seti

Bu çalışmada, askeri hava araçlarının sınıflandırılması amacıyla Military Aircraft Recognition veri seti kullanılmıştır. Bu veri seti, 20 farklı askeri uçak sınıfına ait yüksek çözünürlüklü uydu görüntülerini içermektedir.

2.2 Veri Ön İşleme Süreci

Veri setindeki ham uydu görüntüleri, çoklu nesne içeriğine ve geniş ölçekli arka plan gürültüsüne sahiptir. Modelin öğrenme performansını optimize etmek amacıyla, veri seti nesne tabanlı bir sınıflandırma formatına dönüştürülmüştür.

```
import os
import cv2
import numpy as np
from tqdm import tqdm
import xml.etree.ElementTree as ET

def prepare_mar20_yasa(base_path, split='train', target_size=(64, 64)):
    x, y = [], []

    img_dir = os.path.join(base_path, 'JPEGImages')

    annotations_dir = os.path.join(base_path, 'Annotations', 'Horizontal Bounding Boxes')
    image_set_path = os.path.join(base_path, 'ImageSets', 'Main', f'{split}.txt')

    if not os.path.exists(img_dir):
        print(f"Hata: {img_dir} yolu bulunamadı!")
        return None, None
    if not os.path.exists(annotations_dir):
        print(f"Hata: {annotations_dir} yolu bulunamadı!")
        return None, None
    if not os.path.exists(image_set_path):
        print(f"Hata: {image_set_path} yolu bulunamadı! Lütfen split adını kontrol edin ('train', 'val', 'test').")
        return None, None

    with open(image_set_path, 'r') as f:
        image_filenames = [line.strip() for line in f.readlines()]
```

```

class_map = {}
next_class_id = 0

skipped_image_none = 0
skipped_annotation_missing = 0
skipped_crop_empty = 0
total_processed_objects = 0

for img_name_without_ext in tqdm(image_filenames, desc=f"Uçaklar Kesiliyor ({split})"):
    img_path = os.path.join(img_dir, f'{img_name_without_ext}.jpg')
    annotation_path = os.path.join(annotations_dir, f'{img_name_without_ext}.xml')

    image = cv2.imread(img_path)
    if image is None:
        skipped_image_none += 1
        continue

    h, w, _ = image.shape

    if os.path.exists(annotation_path):
        try:
            tree = ET.parse(annotation_path)
            root = tree.getroot()

            for obj in root.findall('object'):
                class_name = obj.find('name').text
                if class_name not in class_map:

                    class_map[class_name] = next_class_id
                    next_class_id += 1
                    class_id = class_map[class_name]

                bbox = obj.find('bndbox')
                x1 = int(float(bbox.find('xmin').text))
                y1 = int(float(bbox.find('ymin').text))
                x2 = int(float(bbox.find('xmax').text))
                y2 = int(float(bbox.find('ymax').text))

                x1 = max(0, x1)
                y1 = max(0, y1)
                x2 = min(w, x2)
                y2 = min(h, y2)

                crop = image[y1:y2, x1:x2]

                if crop.size == 0 or crop.shape[0] == 0 or crop.shape[1] == 0:
                    skipped_crop_empty += 1
                    continue

                crop = cv2.resize(crop, target_size)
                # Removed grayscale conversion for color images
                X.append(crop) # Append color image directly

                y.append(class_id)
                total_processed_objects += 1
        except Exception as e:
            print(f"Error parsing annotation {annotation_path}: {e}")
            continue
    else:
        skipped_annotation_missing += 1

print(f"\n--- Veri Hazırlığı Özeti ({split}) ---")
print(f"ImageSet dosyasındaki toplam görsel: {len(image_filenames)}")
print(f"cv2.imread'in None döndürdüğü görseller: {skipped_image_none}")
print(f"Eksik etiketleme dosyasına sahip görseller: {skipped_annotation_missing}")
print(f>Kesme işleminin boş görselle sonuçlandığı nesneler: {skipped_crop_empty}")
print(f>Başarıyla işlenen toplam nesne: {total_processed_objects}")
print(f>X içindeki örnek sayısı: {len(X)}")
print(f>y içindeki örnek sayısı: {len(y)}")

return np.array(X), np.array(y)

```



Her bir uçak nesnesinin konumu, XML formatındaki sınırlayıcı kutu (bounding box) verileri kullanılarak OpenCV kütüphanesi ile görüntüden kesilmiştir. Elde edilen tüm nesne görüntüleri, modelin giriş katmanıyla uyumlu olması amacıyla 64 x 64 piksel boyutuna yeniden boyutlandırılmıştır.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def estimate_image_quality(image):
    """
    Bir görüntünün bulanıklık seviyesini Laplacian varyansı ile tahmin eder.
    Daha düşük değerler, daha bulanık görüntülere işaret eder.
    """
    # cv2.cvtColor ve cv2.Laplacian'ın beklediği uint8 formatına dönüştür
    if image.dtype != np.uint8:
        # Eğer görüntü normalize edilmişse (0-1 aralığında), 255 ile çarpıp uint8'e çevirmeliyiz.
        # Mevcut durumda X_train_augmented_balanced 0-255 aralığında float değerler içeriyor.
        image = image.astype(np.uint8)

    # Gri tonlamalıya dönüştür
    gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)
    # Laplacian operatörünü uygula ve varyansını hesapla
    return cv2.Laplacian(gray, cv2.CV_64F).var()

def find_and_visualize_low_quality_images(images, num_samples=10, quality_threshold=100.0):
    """
    Verilen görüntülerden düşük kaliteli olanları tespit eder ve görselleştirir.

    Args:
        images (np.ndarray): Görsellerin numpy dizisi (N, H, W, C).
        num_samples (int): Görselleştirilecek düşük kaliteli örnek sayısı.

        quality_threshold (float): Düşük kalite olarak kabul edilecek Laplacian varyansı eşik değeri.
        Bu değer deneysel olarak ayarlanmalıdır.

    Returns:
        list: Düşük kaliteli olarak tespit edilen görsellerin indeksleri.
        list: Tüm görsellerin kalite skorları.
    """
    quality_scores = []
    for i in range(len(images)):
        score = estimate_image_quality(images[i])
        quality_scores.append(score)

    low_quality_indices = [i for i, score in enumerate(quality_scores) if score < quality_threshold]
    print(f"Toplam {len(low_quality_indices)} adet düşük kaliteli görsel tespit edildi (eşik: {quality_threshold}).")

    if len(low_quality_indices) > 0:
        print(f"İlk {min(num_samples, len(low_quality_indices))} düşük kaliteli görsel örneği gösteriliyor...")
        plt.figure(figsize=(15, 3))
        for i, idx in enumerate(low_quality_indices[:num_samples]):
            ax = plt.subplot(1, num_samples, i + 1)
            plt.imshow(images[idx].astype(np.uint8)) # Görselleştirmede de uint8 kullan
            plt.title(f"Score: {quality_scores[idx]:.1f}")
            plt.axis('off')
        plt.tight_layout()
        plt.show()
    else:
        print("Düşük kaliteli görsel bulunamadı veya eşik değeri çok yüksek.")
```

```
return low_quality_indices, quality_scores
```

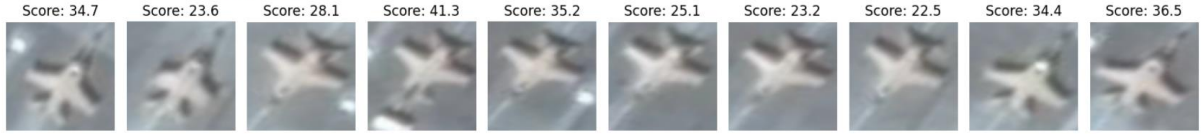
```
# X_combined üzerindeki düşük kaliteli görselleri tespit et
# quality_threshold'u ihtiyaçlarınıza göre ayarlayabilirsiniz.
low_quality_indices_combined, _ = find_and_visualize_low_quality_images(X_combined, quality_threshold=75.0)

# Yüksek kaliteli görsellerin indekslerini bul
high_quality_indices_combined = np.array([i for i in range(len(X_combined)) if i not in low_quality_indices_combined])

# Yeni, filtrelenmiş birleştirilmiş veri setlerini oluştur
X_combined_filtered = X_combined[high_quality_indices_combined]
y_combined_filtered = y_combined[high_quality_indices_combined]

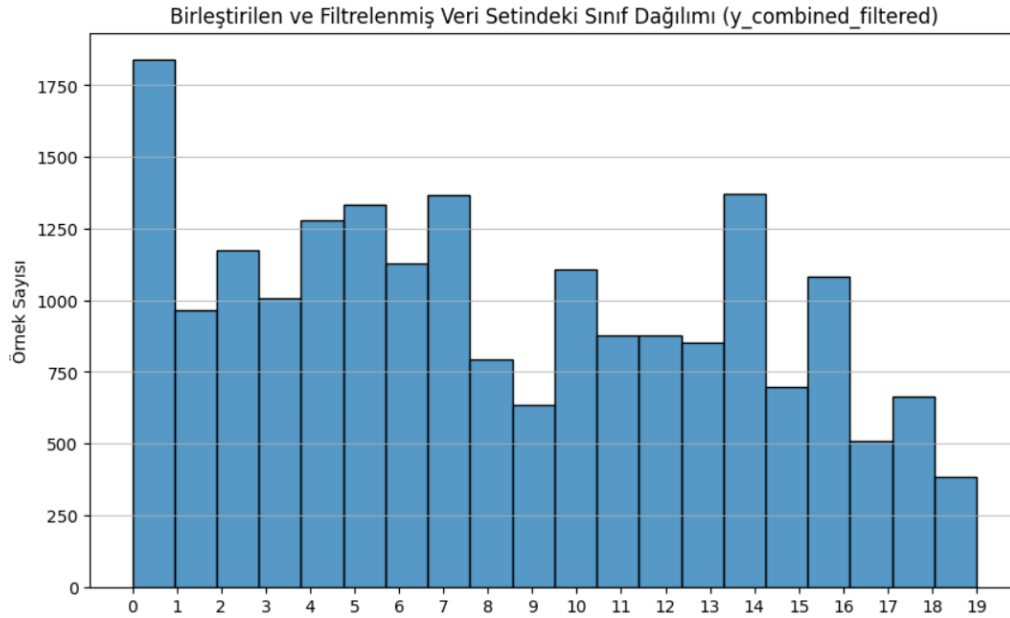
print(f"Orijinal birleştirilmiş veri seti boyutu: {X_combined.shape[0]}")
print(f"Filtrelenmiş birleştirilmiş veri seti boyutu: {X_combined_filtered.shape[0]}")
print(f"Filtrelenmiş X_combined_filtered şekli: {X_combined_filtered.shape}")
print(f"Filtrelenmiş y_combined_filtered şekli: {y_combined_filtered.shape}")
```

Toplam 2394 adet düşük kaliteli görsel tespit edildi (eşik: 75.0).
İlk 10 düşük kaliteli görsel örneği gösteriliyor...



Orijinal birleştirilmiş veri seti boyutu: 22341
Filtrelenmiş birleştirilmiş veri seti boyutu: 19947
Filtrelenmiş X_combined_filtered şekli: (19947, 64, 3)
Filtrelenmiş y_combined_filtered şekli: (19947,)

Derin öğrenme modellerinde eğitim verisinin kalitesi, modelin genelleme başarısı üzerinde doğrudan bir etkiye sahiptir. Uydü görüntülerinde atmosferik koşullar veya sensör hataları nedeniyle oluşabilecek bulanık görüntüler, modelin yanlış örüntüler öğrenmesine neden olabilir. Bu riski minimize etmek amacıyla, görüntü netliğini ölçmek için Laplacian operatörü kullanılmıştır. Laplacian varyansı düşük olan görüntüler, 'düşük kaliteli' olarak sınıflandırılmıştır. Laplacian varyans değeri 75.0 eşliğinin altında kalan, yüksek bulanıklık içeren ve tanımlanması güç olan 2394 adet görsel tespit edilmiştir. Bu görsellerin eğitim sürecinden çıkarılmasıyla, veri seti boyutu 22.341'den 19.947'ye düşürülmüştür.

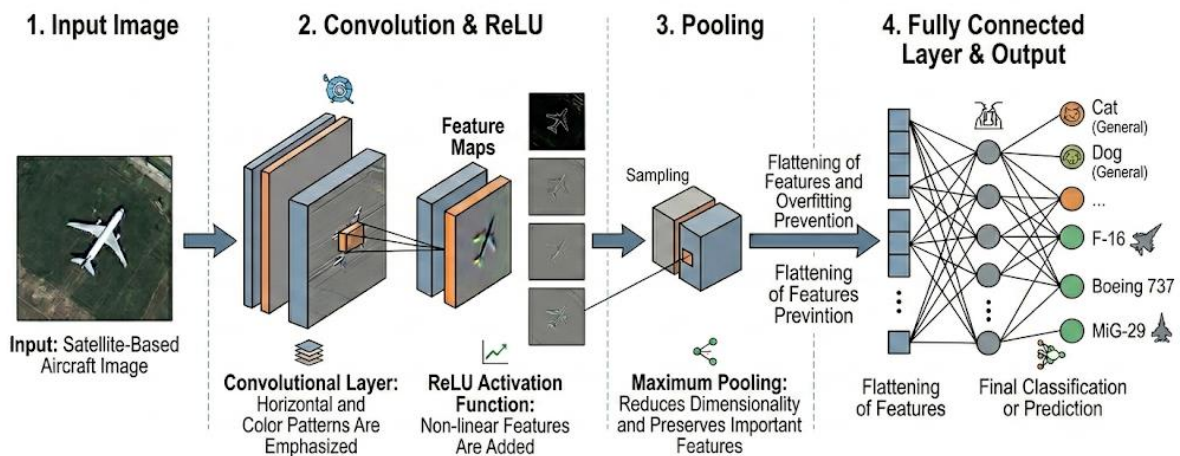


Elde edilen 19.947 adet temizlenmiş görüntüden oluşan veri setinin sınıf dağılımı yukarıdaki grafikte verilmiştir. Veri setindeki örnek sayısı incelendiğinde bazı sınıfların diğerlerine oranla daha yüksek örnek sayısına sahip olduğu gözlemlenmiştir.

2.3 Konvolüsyonel Sinir Ağı (CNN) Mimarisi

Evrişimli Sinir Ağı (CNN), özellikle dijital görüntüler gibi ızgara benzeri bir topolojiye sahip verileri işlemek için tasarlanmış özel bir derin öğrenme mimarisidir. Konvolüsyonel Sinir Ağlarının (CNN) ayırt edici başarısı, karmaşık görsel verileri, anlamsal özellik kaybı olmaksızın daha düşük boyutlu ve işlenebilir öznitelik uzaylarına dönüştürme yeteneğinden kaynaklanmaktadır. Bu, giriş hacmini bir çıktı sınıfına veya değerine dönüştüren belirgin katmanlardan oluşan bir işlem hattı aracılığıyla gerçekleştirilir.

- **Evrişim (Convolutional) Katmanı:** CNN mimarisinin temel yapı taşıdır. Giriş görüntüsü üzerinde gezdirilen öğrenilebilir filtreler aracılığıyla matematiksel evrişim işlemleri gerçekleştirilir. Bu süreç sonucunda oluşturulan öznitelik haritaları, görsel içerikteki yatay/dikey kenarlar ve dokusal gradyanlar gibi temel geometrik örüntüleri temsil eder.
- **Aktivasyon Fonksiyonu:** Evrişim katmanının çıktısına uygulanan doğrusal olmayan bir dönüşüm işlemidir. Literatürde en sık tercih edilen Doğrultulmuş Doğrusal Birim (ReLU) fonksiyonu, negatif değerleri sıfırlayarak ağ içerisinde doğrusal olmayan bir yapı oluşturur. Bu durum, modelin basit doğrusal örüntülerin ötesinde, karmaşık ve hiyerarşik veri yapılarını öğrenmesine olanak tanır.
- **Havuzlama (Pooling) Katmanı:** Alt örnekleme olarak da adlandırılan bu katman, öznitelik haritalarının boyutsal karmaşıklığını düşürür. Özellikle Maksimum Havuzlama (Max-Pooling) gibi tekniklerle bölgedeki en baskın özellikler muhafaza edilirken, parametre sayısı azaltılarak hesaplama verimliliği artırılır ve aşırı öğrenme (overfitting) riski minimize edilir.
- **Tam Bağlantılı (Fully Connected) Katman:** İşlem hattının nihai aşamasını oluşturur. Önceki katmanlardan elde edilen öznitelik haritaları düzleştirilerek (flattening) standart bir yapay sinir ağına aktarılır. Bu katman, çıkarılan yüksek seviyeli özellikleri kullanarak giriş verisinin nihai sınıflandırma veya tahmin işlemini gerçekleştirir.



3. MODEL UYGULAMALARI, PERFORMANS ANALİZİ VE BULGULAR

Bu çalışmanın önceki bölümlerinde tanımlanan veri seti hazırlığı ve derin öğrenme mimarisine yönelik teorik temeller doğrultusunda, bu bölümde uçak sınıflandırma problemini çözmek üzere tasarlanan modellerin deneysel uygulama süreçleri aktarılmaktadır. Çalışma kapsamında, farklı katman derinliklerine ve parametre yapılandırmalarına sahip bir dizi CNN modeli dögüsel bir şekilde geliştirilmiştir. Bu süreçte elde edilen sonuçlar, modelin öğrenme başarısını temsil eden doğruluk (accuracy) ve kayıp (loss) değerleri üzerinden analiz edilmiştir.

3.1 Model 1

Çalışmanın başlangıcında, veri setindeki uçak görüntülerini sınıflandırmak üzere temel bir konvolüsyonel sinir ağı mimarisi tasarlanmıştır. Bu modelin temel amacı, veri seti üzerinde modelin öğrenme kapasitesini test etmek ve sonraki iyileştirmeler için bir referans noktası (baseline) oluşturmaktır.

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras import models
import numpy as np

# Calculate num_classes from the combined original labels, as it represents the total number of unique classes.
# This should be consistent regardless of balancing or splitting.
num_classes = len(np.unique(y_combined))
print(f"Toplam sınıf sayısı: {num_classes}")

# Model tanımı
model = models.Sequential()

# Input shape of the first Conv2D layer corrected to 128x128. Input data (X_train_balanced_normalized)
# will already be in (N, 128, 128, 1) format.
model.add(layers.Conv2D(64, (3, 3), activation='relu', input_shape=(64, 64, 3))) # Changed input_shape to 3 channels
model.add(layers.MaxPooling2D((2, 2)))

model.add(layers.Conv2D(64, (5, 5), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))

model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))

model.add(layers.Conv2D(256, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))

model.add(layers.Flatten())

model.add(layers.Dense(512, activation='relu'))
model.add(layers.Dense(num_classes, activation='softmax'))
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

```
model.summary()
```

Model: "sequential"

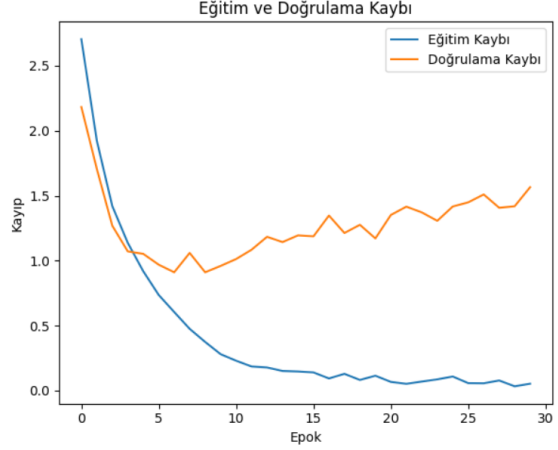
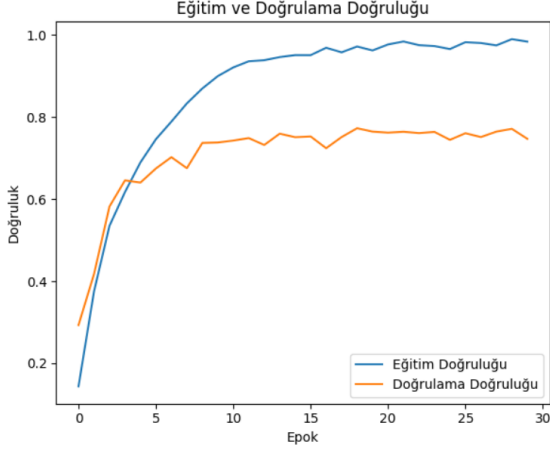
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 62, 62, 64)	1,792
max_pooling2d (MaxPooling2D)	(None, 31, 31, 64)	0
conv2d_1 (Conv2D)	(None, 27, 27, 64)	102,464
max_pooling2d_1 (MaxPooling2D)	(None, 13, 13, 64)	0
conv2d_2 (Conv2D)	(None, 11, 11, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 5, 5, 128)	0
conv2d_3 (Conv2D)	(None, 3, 3, 256)	295,168
max_pooling2d_3 (MaxPooling2D)	(None, 1, 1, 256)	0
flatten (Flatten)	(None, 256)	0
dense (Dense)	(None, 512)	131,584
dense_1 (Dense)	(None, 20)	10,260

Total params: 615,124 (2.35 MB)
Trainable params: 615,124 (2.35 MB)
Non-trainable params: 0 (0.00 B)

Model, toplam 615.124 eğitilebilir parametreden oluşmaktadır.

```
x_train_normalized = x_train_split / 255.0
```

```
history = model.fit(x_train_normalized,  
                    y_train_split,  
                    epochs=30,  
                    batch_size=64,  
                    validation_split=0.2,  
                    verbose=1)
```



Eğitim doğruluğu epok ilerledikçe %100'e yakın bir başarıya ulaşırken, doğrulama (validation) başarımı yaklaşık %75-78 seviyelerinde doygunluğa ulaşmakta ve sabit kalmaktadır. Modelin **aşırı öğrenme (overfitting)** eğilimi gösterdiği gözlemlenmiştir.

3.2 Model 2

3.2.1 Veri Seti Dengeleme ve Arttırma

Eğitim veri setindeki sınıflar arası dağılımın dengesiz olması, modelin baskın sınıflara yönelik önyargılı tahminlerde bulunmasına ve azınlık sınıflarda düşük performans sergilemesine yol açmaktadır. Bu sorunu gidermek amacıyla, her sınıfın 1.000 örnek içerecek şekilde standartlaştırıldığı veri arttırma ve dengeleme stratejisi izlenmiştir.

```

from collections import Counter
import numpy as np
import random # For random.sample

# Now, apply augmentation and balancing only to the training split
# X_train_split is already (N, 64, 64, 3) after previous steps, no need for reshape here
X_train_split_for_augmentation = X_train_split # X_train_split artık filtrelenmiş veriyi içeriyor
y_train_split_for_augmentation = y_train_split # y_train_split artık filtrelenmiş veriyi içeriyor

# Mevcut sınıf dağılımını kontrol et (on X_train_split data)
class_counts = Counter(y_train_split_for_augmentation)
print("Filtrelenmiş Eğitim Veri Setindeki Sınıf Dağılımı (train_test_split sonrası):")
for class_id in sorted(class_counts.keys()):
    print(f"Sınıf {class_id}: {class_counts[class_id]} örnek")

target_samples_per_class = 1000

X_augmented_list = list(X_train_split_for_augmentation) # Listeye çevirerek kolayca ekleme yap
y_augmented_list = list(y_train_split_for_augmentation)

print(f"\nHer sınıf için hedef örnek sayısı: {target_samples_per_class}")

# Upsampling: Sınıfları hedef örnek sayısına kadar arttırma
for class_id in sorted(np.unique(y_train_split_for_augmentation)):
    current_count = class_counts[class_id]
    if current_count < target_samples_per_class:

        num_to_generate = target_samples_per_class - current_count
        print(f"Sınıf {class_id} için {num_to_generate} ek örnek oluşturuluyor...")

        # Sadece bu sınıfa ait görselleri ve etiketleri seç
        class_indices = np.where(y_train_split_for_augmentation == class_id)[0]
        X_class = X_train_split_for_augmentation[class_indices]
        y_class = y_train_split_for_augmentation[class_indices]

        # Veri arttırma jeneratörünü sadece bu sınıfa ait görsellerle besle
        # Ensure that X_class is properly fed to the flow, train_datagen can handle 3 channels
        aug_generator = train_datagen.flow(X_class, y_class, batch_size=1, shuffle=False)

        generated_count = 0
        for _ in range(num_to_generate):
            X_batch, y_batch = next(aug_generator)
            X_augmented_list.append(X_batch[0])
            y_augmented_list.append(y_batch[0])
            generated_count += 1
        print(f"Sınıf {class_id} için {generated_count} örnek başarıyla oluşturuldu.")

X_temp_augmented = np.array(X_augmented_list)
y_temp_augmented = np.array(y_augmented_list)

post_upsample_class_counts = Counter(y_temp_augmented)

X_train_augmented_balanced = []
y_train_augmented_balanced = []

print("\nDownsampling işiemi kontrol ediliyor...")
for class_id in sorted(np.unique(y_temp_augmented)):
    current_count = post_upsample_class_counts[class_id]
    class_indices = np.where(y_temp_augmented == class_id)[0]

    if current_count > target_samples_per_class:
        print(f"Sınıf {class_id} için {current_count} örnek var, {target_samples_per_class} örneğe düşürülecek...")
        sampled_indices = random.sample(list(class_indices), target_samples_per_class)
        X_train_augmented_balanced.extend(X_temp_augmented[sampled_indices])
        y_train_augmented_balanced.extend(y_temp_augmented[sampled_indices])
        print(f"Sınıf {class_id} için {target_samples_per_class} örnek seçildi.")
    else:
        X_train_augmented_balanced.extend(X_temp_augmented[class_indices])
        y_train_augmented_balanced.extend(y_temp_augmented[class_indices])

# Output of this cell should be the balanced and augmented training data
X_train_augmented_balanced = np.array(X_train_augmented_balanced)
y_train_augmented_balanced = np.array(y_train_augmented_balanced)

print(f"\nDengelenmiş ve arttırılmış eğitim veri seti X_train_augmented_balanced boyutu: {X_train_augmented_balanced.shape}")
print(f"Dengelenmiş ve arttırılmış eğitim veri seti y_train_augmented_balanced boyutu: {y_train_augmented_balanced.shape}")

final_class_counts = Counter(y_train_augmented_balanced)
print("\nDengelenmiş ve Arttırılmış Eğitim Veri Setindeki Sınıf Dağılımı:")
for class_id in sorted(final_class_counts.keys()):
    print(f"Sınıf {class_id}: {final_class_counts[class_id]} örnek")

```

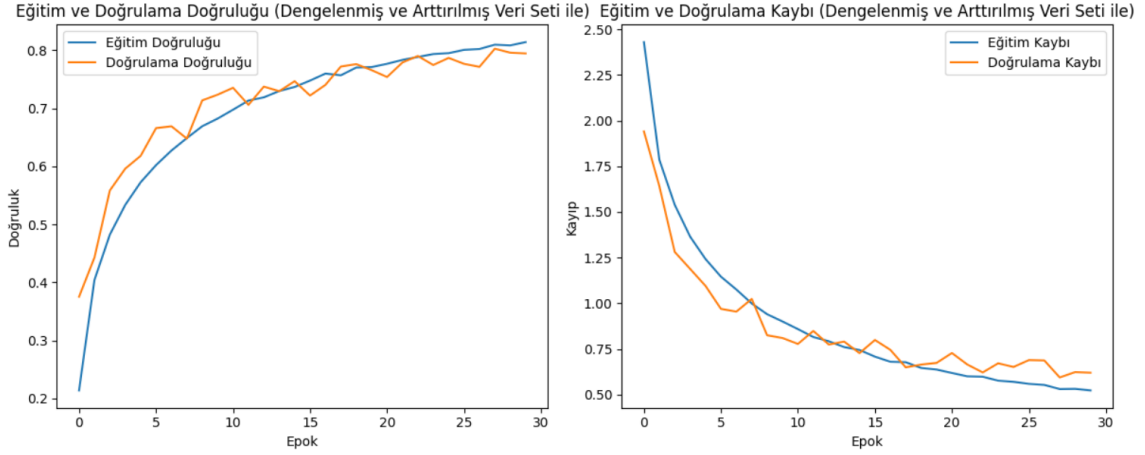
```
# Sadece eğitim setini normalize et (dengelenmiş ve arttırılmış, filtrelenmiş veriden)
X_train_augmented_balanced_normalized = X_train_augmented_balanced / 255.0

# Test setini normalize et (yeni filtrelenmiş test verisi)
X_test_split_normalized = X_test_split / 255.0 # X_test_split artık filtrelenmiş veriyi içeriyor

print(f"Dengelenmiş ve arttırılmış yeni eğitim seti X_train_augmented_balanced_normalized şekli: {X_train_augmented_balanced_normalized.shape}")
print(f"Dengelenmiş ve arttırılmış yeni eğitim seti y_train_augmented_balanced şekli: {y_train_augmented_balanced.shape}")
print(f"Normalize edilmiş filtrelenmiş test seti X_test_split_normalized şekli: {X_test_split_normalized.shape}")
print(f"Filtrelenmiş test seti y_test_split şekli: {y_test_split.shape}")
```

Dengelenmiş ve arttırılmış yeni eğitim seti X_train_augmented_balanced_normalized şekli: (20000, 64, 64, 3)
Dengelenmiş ve arttırılmış yeni eğitim seti y_train_augmented_balanced şekli: (20000,)
Normalize edilmiş filtrelenmiş test seti X_test_split_normalized şekli: (5985, 64, 64, 3)
Filtrelenmiş test seti y_test_split şekli: (5985,)

3.2.2 Eğitim ve Performans Analizi



Dengelenmiş veri seti ve optimize edilmiş mimari ile eğitilen Model 2'nin doğruluk ve kayıp grafikleri incelendiğinde, ilk modelde karşılaşılan aşırı öğrenme eğiliminin, veri setinin dengelenmesi ve stratejik düzenlemelerle büyük ölçüde giderildiği görülmektedir. Model 1, eğitim verisinde %95'in üzerinde bir başarı sergilemesine rağmen doğrulama setinde %75 seviyelerinde bir başarımda kalarak genelleme sorunu yaşarken; Model 2, her iki metrikte de ~%80 seviyelerinde dengeli bir performans sergileyerek gerçek dünya verilerine uyum sağlama noktasında çok daha güvenilir ve tutarlı bir mimari olduğunu kanıtlamıştır.

3.3 Model 3

```
model.summary()
```

Model: "sequential"

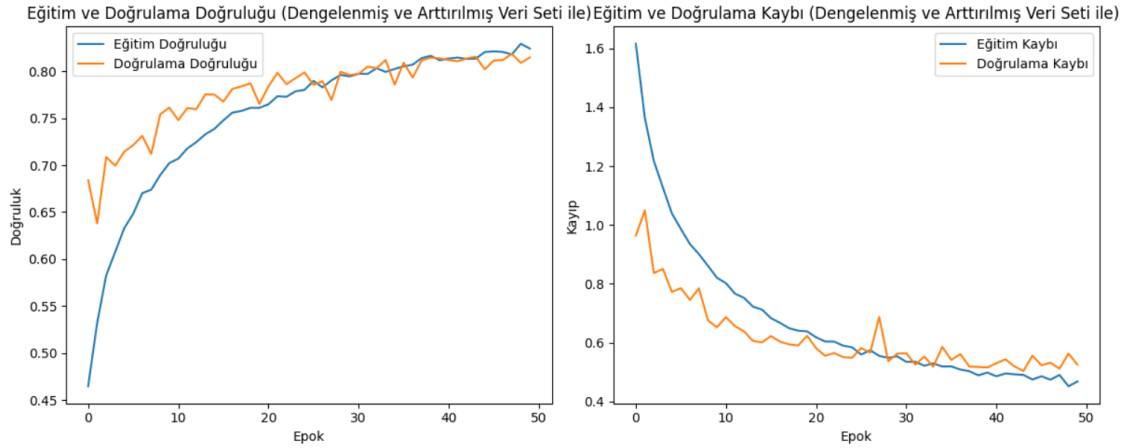
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 62, 62, 64)	1,792
max_pooling2d (MaxPooling2D)	(None, 31, 31, 64)	0
conv2d_1 (Conv2D)	(None, 27, 27, 64)	102,464
max_pooling2d_1 (MaxPooling2D)	(None, 13, 13, 64)	0
conv2d_2 (Conv2D)	(None, 11, 11, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 5, 5, 128)	0
conv2d_3 (Conv2D)	(None, 3, 3, 256)	295,168
flatten (Flatten)	(None, 2304)	0
dense (Dense)	(None, 512)	1,180,160
dense_1 (Dense)	(None, 20)	10,260

Total params: 1,663,700 (6.35 MB)
Trainable params: 1,663,700 (6.35 MB)
Non-trainable params: 0 (0.00 B)

Model 2 ile elde edilen genelleme başarısını daha yüksek bir doğruluk oranına taşımak amacıyla, modelin öğrenme kapasitesini artırmaya yönelik bir mimari güncelleme yapılmıştır. İlk modeldeki son max pooling katmanı modelden çıkartılmış, toplam eğitilebilir parametre sayısı yaklaşık 1,6 milyona yükseltilmiştir.

```
history_balanced_augmented = model.fit(train_datagen.flow(X_train_augmented_balanced_normalized, y_train_augmented_balanced, batch_size=32),
                                       epochs=50,
                                       validation_data=(X_test_split_normalized, y_test_split),
                                       verbose=1)
```

Model 2 ile elde edilen doğrulama grafiği incelendiğinde grafik artışının devam ettiği gözlemlendiği için yeni modelde epochs sayısı 50 olarak belirlenmiştir.



Grafikler incelendiğinde, eğitim ve doğrulama başarımlarının epok boyunca uyumlu bir şekilde artış göstererek yaklaşık %82 seviyesinde olduğu görülmektedir.

3.4 Model 4

```
model.summary()
```

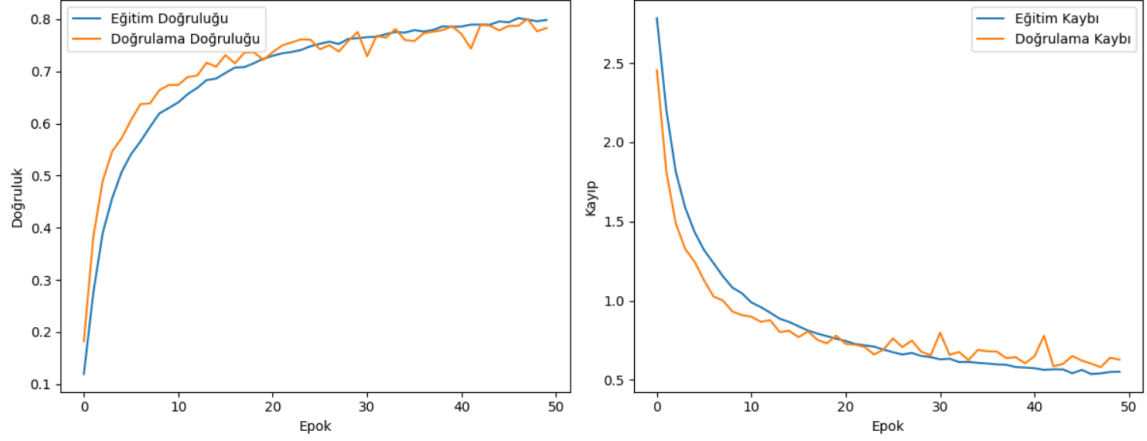
Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 62, 62, 64)	1,792
max_pooling2d_3 (MaxPooling2D)	(None, 31, 31, 64)	0
conv2d_5 (Conv2D)	(None, 27, 27, 64)	102,464
max_pooling2d_4 (MaxPooling2D)	(None, 13, 13, 64)	0
conv2d_6 (Conv2D)	(None, 11, 11, 128)	73,856
max_pooling2d_5 (MaxPooling2D)	(None, 5, 5, 128)	0
conv2d_7 (Conv2D)	(None, 3, 3, 256)	295,168
flatten_1 (Flatten)	(None, 2304)	0
dense_2 (Dense)	(None, 1024)	2,360,320
dense_3 (Dense)	(None, 20)	20,500

Total params: 2,854,100 (10.89 MB)
Trainable params: 2,854,100 (10.89 MB)
Non-trainable params: 0 (0.00 B)

Önceki modellerde elde edilen deneyimler ışığında, ağız öznetelik çıkarma derinliğini ve sınıflandırma kapasitesini optimize etmek amacıyla Model 4 geliştirilmiştir. Bu mimaride, evrişim katmanlarının hiyerarşik yapısı korunurken, son tam bağlantılı (dense) katmanı 1024e çıkartılmıştır. Böylece model toplam 2.854.100 eğitilebilir parametre sayısına ulaşmıştır.

Eğitim ve Doğrulama Doğruluğu (Dengelenmiş ve Arttırılmış Veri Seti ile) Eğitim ve Doğrulama Kaybı (Dengelenmiş ve Arttırılmış Veri Seti ile)



Yapılan mimari genişletme ile gerçekleştirilen eğitim periyodunda, modelin karmaşık ve benzer gövde yapısına sahip uçak sınıflarını daha yüksek bir tutarlılıkla ayırt edebildiği, modelin daha kararlı olduğu gözlemlenmiştir.

3.5 Model 5

model.summary()

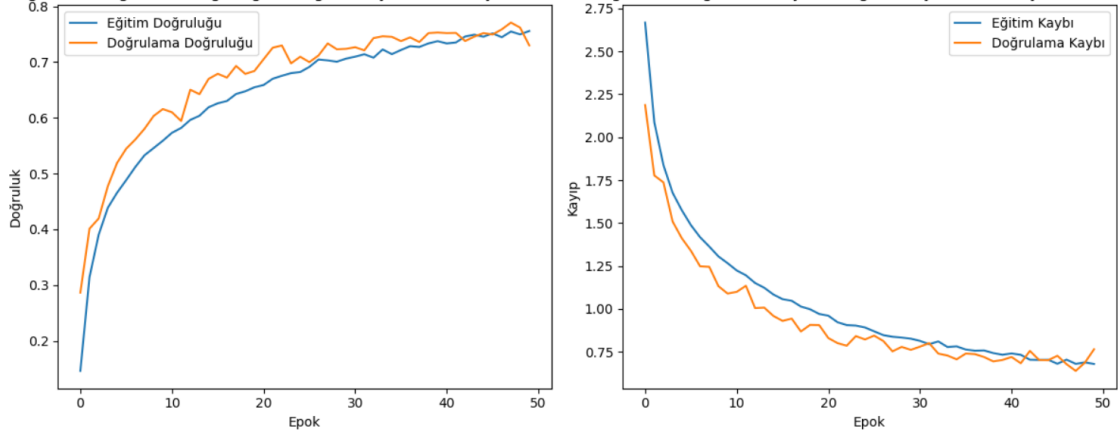
Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 62, 62, 64)	1,792
max_pooling2d (MaxPooling2D)	(None, 31, 31, 64)	0
conv2d_1 (Conv2D)	(None, 27, 27, 64)	102,464
max_pooling2d_1 (MaxPooling2D)	(None, 13, 13, 64)	0
conv2d_2 (Conv2D)	(None, 11, 11, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 5, 5, 128)	0
conv2d_3 (Conv2D)	(None, 3, 3, 256)	295,168
flatten (Flatten)	(None, 2304)	0
dense (Dense)	(None, 2304)	5,310,720
dense_1 (Dense)	(None, 20)	46,100

Total params: 5,830,100 (22.24 MB)
Trainable params: 5,830,100 (22.24 MB)
Non-trainable params: 0 (0.00 B)

Bu mimaride, evrişim katmanlarının hiyerarşik yapısı korunurken, son tam bağlantılı (dense) katmanı 2304e çıkartılmıştır. Böylece model toplam 5.830.100 eğitilebilir parametre sayısına ulaşmıştır.

Eğitim ve Doğrulama Doğruluğu (Dengelenmiş ve Arttırılmış Veri Seti ile) Eğitim ve Doğrulama Kaybı (Dengelenmiş ve Arttırılmış Veri Seti ile)



Parametre sayısının kademeli artışı ile birlikte doğruluğun yaklaşık %80 bandına yükseldiği ve grafiğin kararlı bir doygunluk noktasına ulaştığı gözlemlenmiştir.

3.6 Model 6

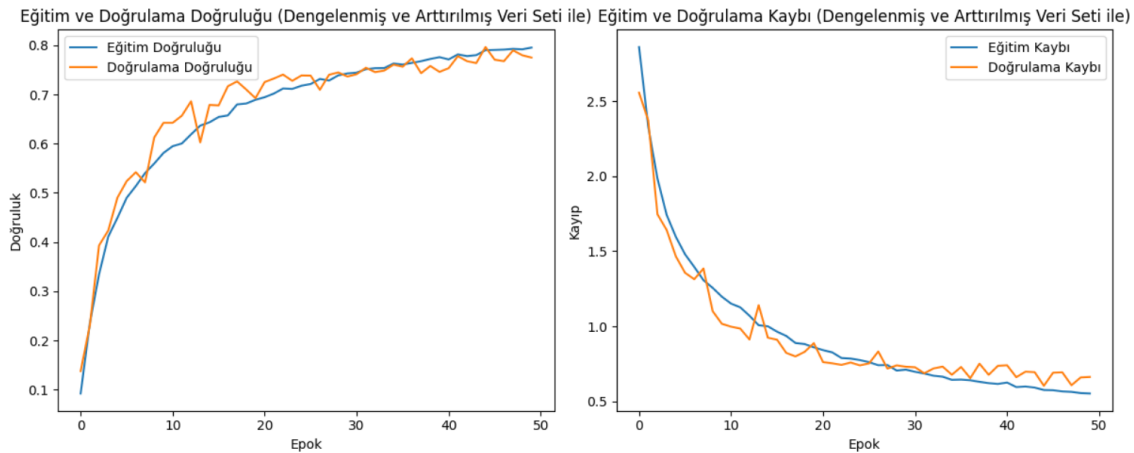
model.summary()

Model: "sequential_3"

Layer (type)	Output Shape	Param #
conv2d_12 (Conv2D)	(None, 62, 62, 64)	1,792
max_pooling2d_9 (MaxPooling2D)	(None, 31, 31, 64)	0
conv2d_13 (Conv2D)	(None, 27, 27, 64)	102,464
max_pooling2d_10 (MaxPooling2D)	(None, 13, 13, 64)	0
conv2d_14 (Conv2D)	(None, 11, 11, 128)	73,856
max_pooling2d_11 (MaxPooling2D)	(None, 5, 5, 128)	0
conv2d_15 (Conv2D)	(None, 3, 3, 256)	295,168
flatten_3 (Flatten)	(None, 2304)	0
dense_6 (Dense)	(None, 1024)	2,360,320
dense_7 (Dense)	(None, 1024)	1,049,600
dense_8 (Dense)	(None, 20)	20,500

Total params: 3,903,700 (14.89 MB)
Trainable params: 3,903,700 (14.89 MB)
Non-trainable params: 0 (0.00 B)

Bu mimaride, evrişim katmanlarının hiyerarşik yapısı korunurken, 2 tane tam bağlantılı (dense) katmanı konulmuştur. Böylece model toplam 3.903.700 eğitilebilir parametre sayısına ulaşmıştır.



Eğitim ve doğrulama sonuçları, modelin eğitim verisi ile doğrulama verisi üzerinde tutarlı bir performans sergilediğini, aşırı öğrenme belirtisi göstermeden sınıflandırma başarısını %80 bandında stabilize ettiğini doğrulamaktadır.

4. TRANSFER ÖĞRENME YAKLAŞIMI İLE PERFORMANS ANALİZİ

Sıfırdan tasarlanan özgün CNN mimarilerinin yanı sıra, ImageNet veri seti üzerinde önceden eğitilmiş VGG16 mimarisi kullanılarak transfer öğrenme yöntemi uygulanmıştır. VGG16'nın derin evrişimli yapısı, görsel hiyerarşiyi önceden öğrendiği için, bu çalışmada öznitelik çıkarıcı olarak konumlandırılmıştır.

Modelin üst katmanları dondurulmuş ve mevcut uçak veri setine uygun olarak yeni sınıflandırma katmanları ile güncellenmiştir.

```
from tensorflow.keras.applications import VGG16
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
import numpy as np

# Calculate num_classes from the combined original labels
num_classes = len(np.unique(y_combined))
print(f"Toplam sınıf sayısı: {num_classes}")

# VGG16 taban modelini yükle
# weights='imagenet' ile önceden eğitilmiş ağırlıkları kullan
# include_top=False ile son katmanları (sınıflandırma katmanları) dahil etme
base_model = VGG16(weights='imagenet', include_top=False, input_shape=(64, 64, 3))

# Taban modeldeki katmanları dondur (ağırlıklarını eğitme)
for layer in base_model.layers:
    layer.trainable = False

# Taban modelin üzerine kendi sınıflandırma katmanlarımızı ekle
x = base_model.output
x = GlobalAveragePooling2D()(x) # Özellik haritalarını tek bir vektöre indirge
x = Dense(256, activation='relu')(x) # Yeni tam bağlı katman
predictions = Dense(num_classes, activation='softmax')(x) # Çıkış katmanı

# Yeni modeli oluştur
transfer_model = Model(inputs=base_model.input, outputs=predictions)

# Modeli derle
transfer_model.compile(optimizer=Adam(learning_rate=0.001),
                      loss='sparse_categorical_crossentropy',
                      metrics=['accuracy'])

print("Transfer öğrenme modeli başarıyla oluşturuldu (VGG16 ile).")

transfer_model.summary()
```

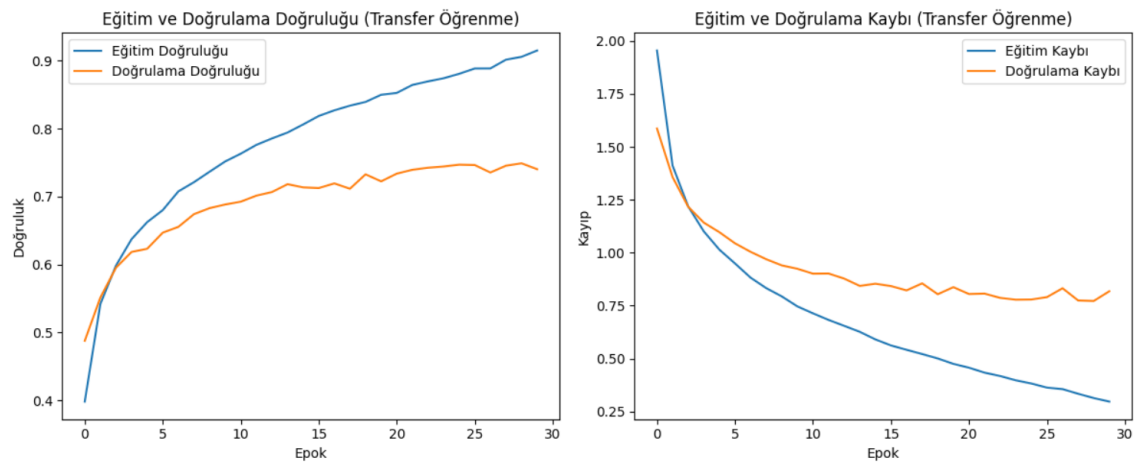
Model: "functional"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 64, 64, 3)	0
block1_conv1 (Conv2D)	(None, 64, 64, 64)	1,792
block1_conv2 (Conv2D)	(None, 64, 64, 64)	36,928
block1_pool (MaxPooling2D)	(None, 32, 32, 64)	0
block2_conv1 (Conv2D)	(None, 32, 32, 128)	73,856
block2_conv2 (Conv2D)	(None, 32, 32, 128)	147,584
block2_pool (MaxPooling2D)	(None, 16, 16, 128)	0
block3_conv1 (Conv2D)	(None, 16, 16, 256)	295,168
block3_conv2 (Conv2D)	(None, 16, 16, 256)	590,080
block3_conv3 (Conv2D)	(None, 16, 16, 256)	590,080
block3_pool (MaxPooling2D)	(None, 8, 8, 256)	0
block4_conv1 (Conv2D)	(None, 8, 8, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 8, 8, 512)	2,359,808

block4_conv3 (Conv2D)	(None, 8, 8, 512)	2,359,808
block4_pool1 (MaxPooling2D)	(None, 4, 4, 512)	0
block5_conv1 (Conv2D)	(None, 4, 4, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 4, 4, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 4, 4, 512)	2,359,808
block5_pool1 (MaxPooling2D)	(None, 2, 2, 512)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 512)	0
dense (Dense)	(None, 256)	131,328
dense_1 (Dense)	(None, 20)	5,140

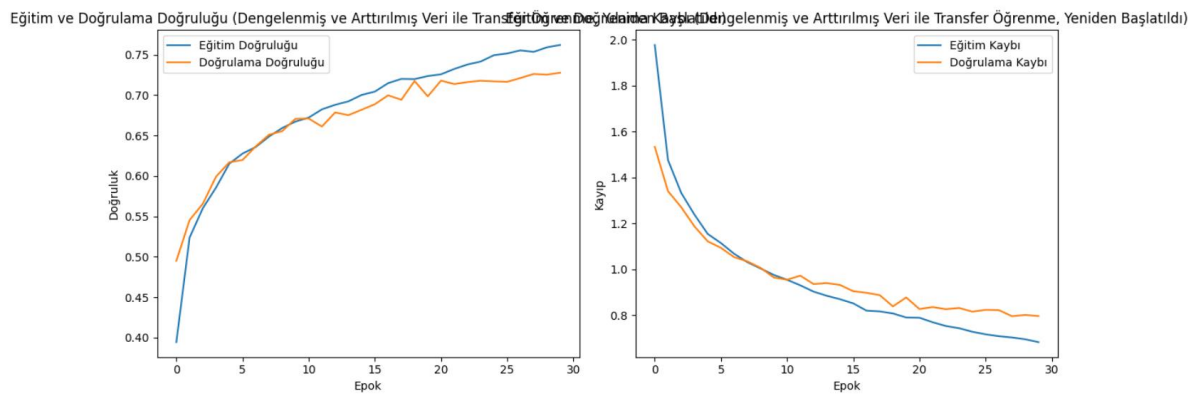
Total params: 14,851,156 (56.65 MB)
Trainable params: 136,468 (533.08 KB)
Non-trainable params: 14,714,688 (56.13 MB)

4.1 Ham Veri Seti ile VGG16



VGG16 mimarisi, ham veride hızlı öğrenme yeteneği sergilemiş ancak sınırlı veri setlerinde aşırı öğrenme (overfitting) eğilimi göstermiştir.

4.2 Dengelenmiş ve Arttırılmış Veri Seti ile VGG16



Dengelenmiş ve artırılmış veri seti ile yapılan eğitimde, doğrulama eğrisinin daha kararlı bir seyir izlediği ve kayıp fonksiyonundaki dalgalanmaların minimize edildiği gözlemlenmiştir.

5. SONUÇ

Bu çalışma kapsamında, derin öğrenme yöntemleri kullanılarak uçak modellerinin otomatik sınıflandırılması hedeflenmiştir. Deneysel süreçte, sıfırdan tasarlanan özgün evrişimli sinir ağı (CNN) mimarileri ve akademik literatürde yer etmiş önceden eğitilmiş VGG16 mimarisi ile transfer öğrenme teknikleri karşılaştırmalı olarak analiz edilmiştir.

Yapılan çalışmalar neticesinde aşağıdaki temel sonuçlara ulaşılmıştır:

- **İterasyonun Önemi:** Model başarısının sadece mimari derinlikle değil, veri setinin niteliği ile doğrudan ilişkili olduğu kanıtlanmıştır. İteratif iyileştirme süreci, modelin veriyi ezberlemesi (overfitting) yerine genel geçer öznitelikleri öğrenmesini sağlamıştır.
- **Model Kapasitesi:** Kurulan CNN mimarileri, uçak sınıflarına ait özgün yapısal öznitelikleri (gövde yapısı, kanat geometrisi vb.) başarılı bir şekilde ayrıştırarak, donanım verimliliği ile yüksek başarı oranını dengeli bir şekilde sunmuştur.
- **Transfer Öğrenme ve Veri İlişkisi:** VGG16 mimarisi, yüksek parametre yoğunluğu sayesinde karmaşık örüntüleri yakalamada başarılı olmuş ancak sınırlı veri setlerinde "veri kalitesinin" model mimarisinden daha belirleyici olduğu gözlemlenmiştir. Ham veride sergilenen yüksek başarı oranlarının, aslında bir ezberleme (overfitting) sonucu olduğu, ancak dengelenmiş veri seti ile modelin gerçek genelleme başarısına ulaştığı kanıtlanmıştır.

Bu çalışma sadece uçakları sınıflandıran bir model sunmamakta; aynı zamanda verinin niteliği, model mimarisinin karmaşıklığı ve transfer öğrenme yöntemlerinin performans üzerindeki etkilerini deneysel bir süreçle analiz etmektedir. Elde edilen bulgular, model başarısının sadece derinlik veya parametre sayısı ile değil, verinin doğru işlenmesi (dengeleme/artırma) ile doğrudan ilişkili olduğunu kanıtlamaktadır.

6. KAYNAKÇA

- Veri Seti: <https://www.kaggle.com/datasets/khlaifiabile/military-aircraft-recognition-dataset>
- <https://www.ultralytics.com/tr/glossary/convolutional-neural-network-cnn#evri%C5%9Fimli-sinir-a%C4%9Flar%C4%B1-nas%C4%B1l-%C3%A7al%C4%B1%C5%9F%C4%B1r>
- Google. (2026). *Gemini* (3.1) [Yapay zeka dil modeli]. google.com